

Databricks Platform Security Docs

Question: What is the security posture of Databricks Genie Code Agent mode?



Table of Contents

Overview	3
Security Model at a Glance	3
Platform Boundaries	4
Identity and permissions	4
Compute and network controls	5
Data used by Agent Mode	5
Tools and customization surfaces	5
Audit logging and administrative visibility	6
Execution Safety Architecture	6
Tool and action routing	6
Deterministic policy	7
AI code and command safety inspection	7
Human approval and saved preferences	7
Intent-based Auto-approve	8
Execution under platform enforcement	9
Model, Content, and Prompt Integrity Controls	9
Provider and Databricks content controls	9
Server-side abuse protection	10
Prompt, routing, and retrieved-content integrity	10
Assurance and Continuous Improvement	10
What This Means for Customers	11
Shared Controls for Databricks AI Features	12
Revision history	12

Overview

[Genie Code](#) is the autonomous, multi-step capability within Databricks' in-product coding assistant. From a natural-language request, it can plan work, retrieve relevant assets, inspect metadata and code context, write code, use tools, run approved actions, and iterate on execution results.

The security model is built around bounded agency. Genie Code does not receive a separate or elevated identity, and no model classifier is treated as the security boundary. Actions execute as the authenticated user and remain subject to the same Unity Catalog permissions, workspace controls, compute context, network policies, and audit mechanisms that govern the user's direct activity.

When a proposed action can create side effects, it enters a layered pre-execution path. Tool classification first determines whether the operation is read-only or side-effecting. Deterministic policy handles known-safe and known-dangerous cases. AI safety inspection analyzes commands that require semantic judgment. A remaining action is then passed for the user's chosen approval strategy, either human review of each individual action or intent-based Auto-approve (see [Why Auto-approve?](#) to understand the security benefits of this approach). Only after those checks can the action execute under the user's identity.

This design separates three concerns that are sometimes conflated in discussions of agent security:

- **Authority:** What the user is permitted to access or change. Unity Catalog and platform permissions enforce this boundary.
- **Action safety:** Whether a proposed command should proceed. Deterministic policy, safety inspection, and approval govern this decision.
- **Model interaction safety:** How prompts, retrieved content, and completions are filtered and protected. Provider and Databricks-managed controls operate at this layer.

The result is a defense-in-depth architecture in which prompt content does not redefine platform permissions, Auto-approve does not replace access control, and execution remains attributable to the user who initiated it.

This paper is focused on Genie Code Agent Mode. Before Agent Mode there was a simpler chat-only version, which had fewer capabilities. This paper extends the broader Databricks AI Features Security and Governance reference paper, which is available via your Databricks account team.

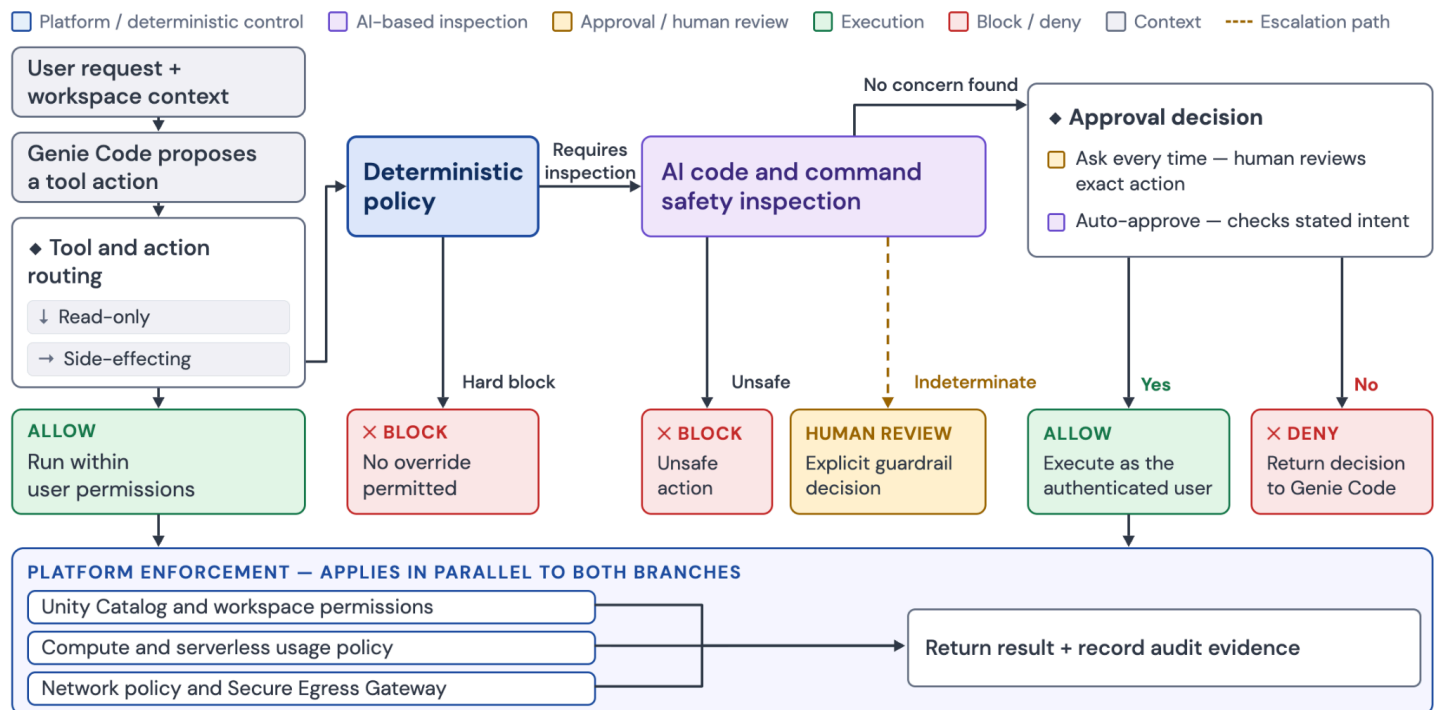
Security Model at a Glance

The model can propose an action, but the platform determines what can execute. The most important security property is the composition of independent controls rather than reliance on a single classifier.

Platform boundary. Authentication, Unity Catalog, workspace permissions, compute policy, and network policy determine the user's available authority. The agent cannot grant itself additional privileges.

Pre-execution boundary. Side-effecting actions pass through deterministic routing, policy checks, AI safety inspection, and an approval strategy. Known destructive operations can be hard-blocked before any approval model is consulted.

Execution boundary. Approved actions execute in the user's compute and identity context. Platform authorization is evaluated at execution time, and activity is logged using the standard Databricks mechanisms.



Platform Boundaries

Authentication, Unity Catalog, workspace permissions, compute policy, and network policy determine the user's available authority. The agent cannot grant itself additional privileges.

Identity and permissions

Genie Code runs actions as the authenticated user. It does not operate with a shared agent credential, an administrator role, or an additional set of hidden grants. Reading metadata, sampling a table, running a notebook cell, executing SQL, validating a pipeline, or creating a platform object is evaluated in the context of the user who initiated the task.

Unity Catalog remains the authorization boundary for governed data and AI assets. Data supplied to the model respects the user's Unity Catalog permissions. At execution time, an attempted read or write that exceeds the user's grants fails in the same way as a comparable action initiated directly by that user.

This is a foundational control because it holds independently of the model's behavior. A generated command can request broader access, but a request is not a grant. The platform still authorizes the operation.

Compute and network controls

Execution uses the compute associated with the user's context. Genie Code can use serverless compute for supported actions and can use compute already selected in the product surface. In either case, code runs within the workspace's configured environment and policies.

Network controls continue to apply. When [Serverless Egress Control](#) is configured, outbound traffic remains subject to those rules regardless of whether the underlying code was typed by a person or proposed by Genie Code. An agent proposal does not create an alternate network path.

Data used by Agent Mode

Genie Code sends only the context needed to provide the feature. Databricks sends your prompt, relevant table metadata and values, errors, as well as input code or queries to help return more relevant results. Agent Mode can analyze cell outputs and other execution feedback so that it can correct errors and continue a multi-step task.

All data supplied to the model respects Unity Catalog permissions. Partner-powered AI assistive features use zero-data-retention endpoints from model partners, and model partners do not store prompts or responses submitted through those routes. Databricks does not use customer prompts, data, or responses from these features to train generative foundation models offered to third parties. See [Databricks AI assistive features trust and safety](#) for the current service-specific details.

Traffic to model providers is encrypted in transit. Databricks-side content is encrypted at rest, and customer-managed-key coverage applies where supported and enabled. Current scope is documented under [customer-managed keys](#).

Tools and customization surfaces

Genie Code can use tools for asset discovery, notebook and SQL execution, file operations, visualization, Databricks CLI workflows, and other product-surface capabilities. Available tools vary by surface and configuration. Tool availability does not itself grant permission: the user must still be authorized to perform the underlying operation.

Customization introduces additional context and capabilities, with an explicit owner for each trust decision:

- [Instructions](#) provide persistent context such as organizational conventions, preferred libraries, and coding guidelines. Workspace administrators control workspace instructions; users control their own instructions. Instructions guide model behavior but are not executable permission grants.
- [Skills](#) package domain knowledge, workflows, supporting resources, and optionally executable scripts. Workspace skills are shared and administered at workspace scope; user skills remain private to an individual user. Executed skill code runs with the user's permissions.
- [MCP servers and connectors](#) connect Genie Code to external tools and content. Workspace administrators control which servers are available, and users select from sources they are permitted to use. For native third-party connectors, users authenticate individually and OAuth tokens are not shared between users.

Administrators should treat executable skills, external MCP servers, and connected tools as software entering the user's trust boundary. Their permissions, deployment location, credentials, and network destinations should be reviewed using the same principles applied to other workspace integrations.

[Audit logging and administrative visibility](#)

Actions performed through Genie Code are attributable to the authenticated user in the relevant Databricks audit records. There is no elevated agent actor that obscures who initiated the work. The specific audit event depends on the underlying operation, such as a query, workspace change, job action, or data-access event.

For adoption and interaction visibility, Databricks provides the `system.access.assistant_events` [Genie Code system table](#). Each row represents a user-submitted Genie Code message and includes identifiers such as account, workspace, time, and initiating user. Security and platform teams can correlate these records with the audit events created by the underlying actions. Customers desiring more detailed visibility can be connected with our product management team.

Execution Safety Architecture

Agent Mode's execution-safety path is designed to prevent a model recommendation from becoming an unchecked side effect. Each stage has a narrower responsibility, and later platform enforcement remains in place even when earlier checks allow an action.

[Tool and action routing](#)

The first stage identifies the tool and type of action. Read-only inspection can proceed with lower friction, while operations that execute code, modify data, edit assets, create resources, or change configuration enter the safety and approval path.

This classification is deterministic. Content retrieved by the agent cannot persuade the router to treat a side-effecting tool as read-only. Routing establishes which downstream checks are mandatory; it is not itself the full safety decision.

Deterministic policy

Rules-based policy settles cases where a clear, stable decision can be made without model judgment. Known-safe read-only commands can be fast-tracked. Known-dangerous classes of operation can be blocked with no Auto-approve or saved-preference override.

Representative hard-block categories include destructive deletion of selected compute, job, pipeline, workspace, catalog, credential, identity, database, or deployment resources; commands containing unresolved templates; and commands that reference credential-bearing environment variables. The exact catalog evolves as product capabilities and threat models change.

Deterministic checks are deliberately precise rather than universal. They provide strong handling for known command patterns. Operations requiring semantic interpretation continue to the safety-inspection layer.

AI code and command safety inspection

For actions not conclusively handled by deterministic policy, dedicated safety models inspect the exact proposed code, command, or tool arguments. They look for behavior such as:

- Destructive SQL or DDL, broad deletes, and high-impact schema changes
- Permission grants, revocations, or identity and secret changes
- Data or credential exfiltration to external destinations
- Remote-code-execution patterns and unsafe package installation
- Production deployments or changes that alter the code run by a job or pipeline
- Destructive filesystem or workspace-file mutations
- External API calls and source-control actions with an excessive blast radius

Safety inspection is fail-safe in its routing. An unavailable, malformed, or indeterminate classifier result is not silently converted into approval. The action is blocked or moved to an explicit guardrail decision path, depending on the tool and operation.

Human approval and saved preferences

The default approval strategy is **Ask every time**. The user sees the proposed tool action and can allow or refuse it. This keeps a human in the loop for code execution and persistent changes.

Genie Code also supports **Allow in current chat** and **Always allow** preferences for eligible actions. These settings reduce repeated prompts; they do not disable deterministic policy, platform authorization, network controls, or safety handling for actions that require a stronger decision. Saved preferences are evaluated after the earlier safety stages rather than replacing them.

Intent-based Auto-approve

Why Auto-approve?

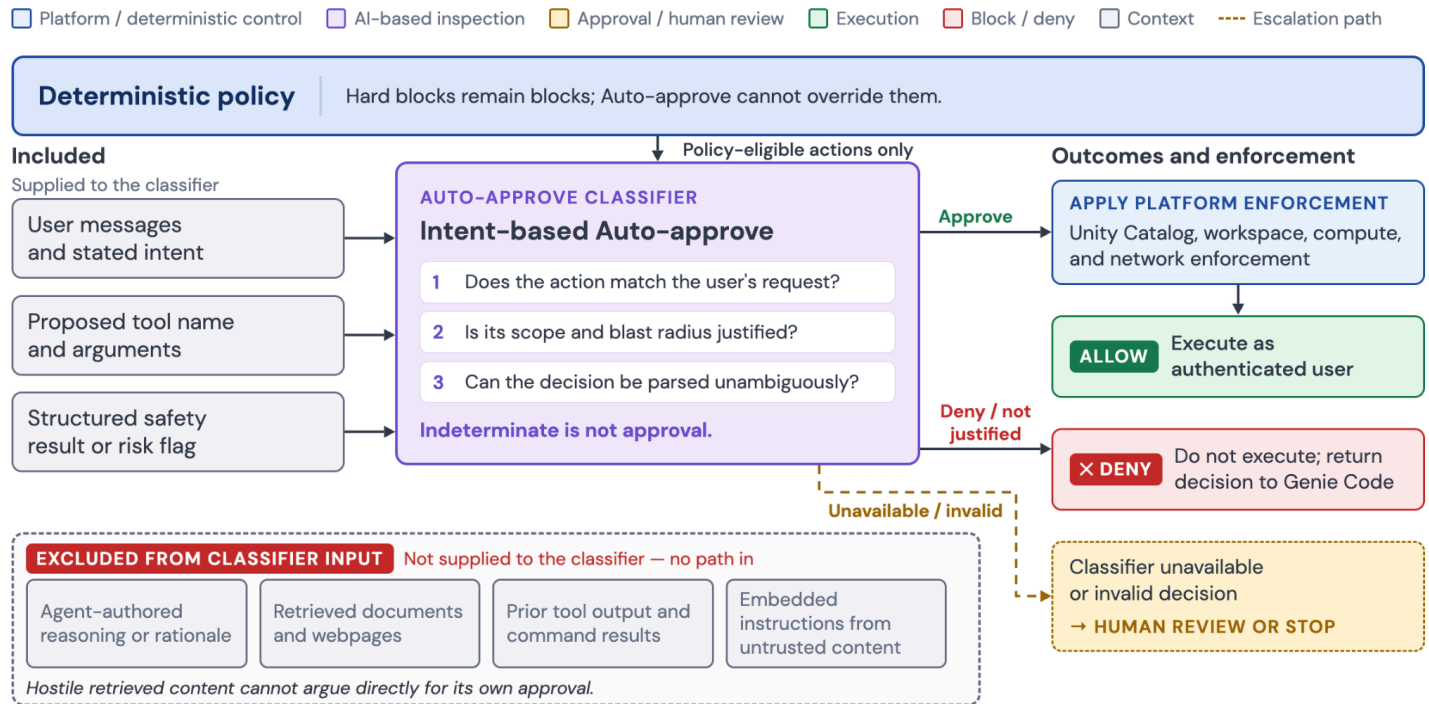
When coding agents first became popular, they routinely made mistakes, and users were required to review each command before it was executed to ensure safety. But this model has not scaled well to a world where agents are substantially more reliable, capable of taking more actions in the course of a single request, and capable of accomplishing so much more. Even if a human user wished to be in the loop on all actions in all sessions, in practice humans quickly click approve on everything, without much review at all.

Auto-approve is intended to strengthen the practical approval boundary for regular Genie Code users, not to remove it. Genie Code Auto-approve replaces repeated human prompts with an AI classifier that evaluates a proposed action for safety and alignment with the user's stated intent.

This approach is now standard across the industry. In an [August 2026 analysis of classifier-mediated Auto mode in Claude Code](#), Anthropic reports that human reviewers caught 13.6% of inserted dangerous commands, whereas their Auto mode caught 89%. Both Anthropic and Databricks still recommend human review for high-stakes production changes, but in day-to-day actions, Auto mode takes advantage of the model's ability to closely review proposed actions without fatigue or inattention.

How does Auto-approve work?

Auto-approve is designed not just to check whether a command is generally safe, but whether its scope and effect are justified by what the user asked Genie Code to do. A request to summarize a notebook, that does not justify dropping a production table. A request to delete a narrow set of records does not justify truncating an entire dataset. A request to inspect a job does not justify changing its production code source. Applying this judgement addresses core security risks alongside the potential for model hallucination (or models that are too helpful).



The approval classifier receives a constrained view: the user's messages and the proposed tool arguments needed for the decision. Agent-authored reasoning and arbitrary content retrieved from documents, webpages, notebooks, or tool output are excluded from the judge's decision context. This reduces the ability of hostile retrieved content to argue itself into approval.

Auto-approve enforced limitations

Auto-approve remains subordinate to deterministic policy and platform enforcement. It cannot grant permissions, open a blocked network destination, or convert a deterministic denial into an allowed action. If the classifier cannot make a valid decision in an interactive flow, the system can fall back to human approval rather than silently proceed. Auto-approve is an approval strategy, not a replacement for security, compliance, or access-control requirements.

Execution under platform enforcement

After an action passes the applicable checks and approval strategy, it executes as the authenticated user in the selected compute context. Unity Catalog, workspace authorization, compute policy, serverless usage policy, network policy, and Secure Egress Gateway controls apply at execution time. The resulting platform action is logged using the standard Databricks mechanisms.

Model, Content, and Prompt Integrity Controls

The execution safety controls above determine whether tool actions can run. Content and prompt-integrity controls protect the model interaction itself. These layers address different risks and are designed to complement one another.

Provider and Databricks content controls

Databricks applies input and output protections according to the model and serving route. Azure-hosted model traffic can use Azure Responsible AI content filtering. Anthropic routes use Anthropic's built-in safety mechanisms and Databricks hardening. Where Amazon Bedrock is part of the serving route, Bedrock Guardrails can provide an additional provider layer. Databricks-managed abuse-prevention guidance and classifiers add product-specific controls.

Depending on the provider, policy, and feature, input checks can cover violence, hate, sexual content, self-harm, direct jailbreak attempts, indirect prompt injection, instruction override, and request-origin validation. Output checks can cover harmful-content categories, unsafe code patterns, and protected text or code material.

Filter policy is managed by Databricks at the feature level. Certain Azure-hosted routes can support customer-specific content-filter configurations; equivalent customization is not uniform across all model providers. Customers should confirm current availability for their serving route with their Databricks account team.

Content filters do not replace action controls. A completion can pass a harmful-content filter and still be blocked by deterministic policy, safety inspection, user approval, Unity Catalog, or network enforcement.

Server-side abuse protection

Databricks applies server-side protections intended to keep Genie Code within legitimate product use. These protections can detect attempts to override the service's role, use the service as an unrestricted general-purpose model endpoint, or generate abusive and off-topic workloads.

Because this enforcement occurs in the Databricks service layer, it does not depend solely on the browser or a particular UI path. It also remains logically separate from command safety: prompt-level abuse controls govern use of the AI service, while execution controls govern side effects in the workspace.

Prompt, routing, and retrieved-content integrity

Databricks adds product, feature, and safety instructions to model requests. End-user prompt text does not directly control internal model-routing fields or remove service-injected policy. Model selection and routing remain controlled by Databricks so that geo, feature, and account policy are not bypassed by merely naming a model in a prompt.

Databricks constructs protected prompt fragments—product instructions, safety addenda, and feature context—separately from user-controlled prompt text. These service-injected fragments are placed in the system-instruction context with precedence over caller-supplied instructions. Origin validation distinguishes a Databricks-issued system context from a user message that merely claims Databricks authority, while internal routing and control fields are removed before forwarding. Together, these controls help prevent ordinary prompt text or retrieved content from replacing the service's protected instructions or redefining how a request is routed.

Retrieved documents and tool responses are treated as untrusted content rather than as new user authority. Where applicable, tool output is scanned for instruction-override and prompt-injection patterns before it is returned to the agent's working context. The constrained Auto-approve design adds another separation: retrieved content is not supplied to the intent classifier that approves a tool action.

Assurance and Continuous Improvement

Agentic threats change as capabilities expand, customers develop new workflows, and attackers develop new prompt-injection or action-abuse techniques. The guardrail system is therefore maintained as an evolving security control set rather than a one-time model prompt.

Databricks uses several complementary assurance mechanisms:

- **Security design review:** Agent capabilities and major approval-mode changes undergo platform security and AI-security review before release.
- **Deterministic regression tests:** Policy routing, hard blocks, approval precedence, and fail-safe behavior are tested as code and can block changes from merging.
- **Model-based evaluations:** Safety classifiers are evaluated against representative safe and unsafe actions across SQL, shell, Python, REST, CLI, jobs, pipelines, credentials, external calls, and source control.
- **End-to-end tests:** Browser and service tests verify the approval contract, including negative cases such as ensuring a declined action does not execute through an alternate path.
- **Operational controls:** Restricted ownership, production monitoring, feature flags, and runtime kill switches support controlled rollout and rapid response.
- **Trace-informed improvement:** Completed traces can be reviewed offline to find actions that exceeded apparent intent, carried excessive blast radius, or revealed a new attack pattern.

Evaluation coverage is organized around threat categories rather than a single aggregate score. Representative categories include data destruction, credential theft, exfiltration, prompt injection, permission escalation, unsafe external calls, remote code execution, source-control danger, job and pipeline mutation, cost abuse, and false-positive regression for safe read-only or explicitly authorized actions.

When a gap is identified, remediation is selected at the appropriate layer. The answer may be a deterministic block, tighter approval routing, an additional platform permission check, improved untrusted-content handling, clearer product behavior, or expanded evaluation coverage—not only a change to a model prompt.

What This Means for Customers

Customer concern	Primary controls
Can the agent gain broader data access?	Actions run as the authenticated user; Unity Catalog and workspace permissions remain authoritative.
Can hostile content cause an unsafe action?	Retrieved content is treated as untrusted; side effects pass through deterministic policy, safety inspection, and approval; Auto-approve does not receive arbitrary retrieved content.
Can the agent perform destructive operations unattended?	Selected destructive operations are hard-blocked; other risky commands receive safety inspection and an approval decision before execution.
Can code exfiltrate data?	Safety inspection looks for external calls and exfiltration behavior; Serverless Egress Control (and classic compute equivalents) enforce allowed destinations at runtime.
Does Auto-approve bypass security policy?	No. It evaluates intent after earlier checks and remains subordinate to deterministic rules, Unity Catalog, workspace authorization, and network controls. Its goal is to improve security by dissuading users suffering approval fatigue from approving everything without looking.
Can administrators attribute activity?	Underlying actions are logged as the authenticated user; <code>system.access.assistant_events</code> provides Genie Code interaction metadata for monitoring and correlation.
Do model providers retain customer prompts?	Partner-powered routes use zero-data-retention endpoints, and model partners do not store prompts or responses submitted through those routes.

Shared Controls for Databricks AI Features

This paper focuses on the controls introduced or made more visible by agentic execution. Genie Code also inherits the broader security and governance controls applied to Databricks AI assistive features.

- **Confidentiality and training:** Databricks does not use customer prompts, data, or responses from AI assistive features to train generative foundation models made available to third parties. Partner model providers do not retain prompts or responses submitted through partner-powered routes.
- **Encryption:** Traffic to model providers is encrypted in transit. Databricks-side stored content is encrypted at rest, with customer-managed-key coverage where supported and enabled.

- Data minimization: The service sends the prompt and only the context needed for the feature, subject to the user's Unity Catalog permissions.
- Residency: Databricks AI assistive features are Designated Services and use Databricks Geos to manage data-residency boundaries. Cross-Geo processing is governed by the workspace's configuration and feature availability.
- Controlled Databricks access: Access to production systems and retained customer content is governed by Databricks production-access controls and centralized security logging.
- Compliance: Requirements for regulated data depend on the workspace configuration and applicable compliance standard. See the current [trust and safety documentation](#) for supported configurations.

For current product behavior, availability, and compliance scope, refer to the [Genie Code documentation](#), [partner-powered AI features](#), and the [Databricks Security and Trust Center](#).

Revision history

Date	Author	Description of changes	Version #
05/11/26	JB	First version	1.0
08/09/26	DV	Reframed the security architecture and incorporated updated execution guardrails, Auto-approve, content controls, and assurance practices	2.0